SIMATS ENGINEERING

ASSESSMENT TOOL II – REVERSE ENGINEERING TASK (MEDIUM)

Course: Ethical Hacking for Secure Networks (ITA1436) | CO2

Course Name: Ethical Hacking for Secure Networks	Code: ITA1436 CO: CO2
Duration: 90 min	Total Marks: 50
Date: 5.8.2026	Reg. No / Name: AJAY U 192421237

CO2: Execute footprinting in social engineering such as DNS analysis, competitive intelligence gathering, and attack simulations to collect relevant information for identifying potential security risks. (BL3)

Task 1: FootPrintX.exe – Footprinting Toolkit Analysis

FootPrintX.exe is presented as a reconnaissance utility that gathers domain names, IP addresses, and network details about a target organisation. Static and dynamic reverse-engineering analysis (disassembly, string extraction, and API-call monitoring) of such a binary typically reveals the following.

1. Which functions or modules are responsible for gathering target information?

Disassembly of a footprinting binary of this kind usually shows a modular design in which each information-gathering activity is wrapped in its own function, all invoked from a central driver/main routine:

- ResolveTarget() / DNSLookup() – calls system resolver APIs (e.g., gethostbyname, DnsQuery on Windows or getaddrinfo) to convert the supplied domain into IP addresses.
- WhoisQuery() – opens a raw TCP socket to a WHOIS server (port 43) or calls an HTTP-based WHOIS API to pull registrant, registrar, and name-server information.
- PortProbe() / NetworkScan() – iterates over a list of common ports using connect() or SYN-style socket calls to fingerprint open services on the resolved IPs.
- BannerGrab() – reads the initial response from any open port (HTTP headers, SSH banners) to infer running services and versions.
- ReportBuilder() / LogWriter() – aggregates the results from the above modules into a structured output file (CSV/JSON/TXT) that the tool saves locally or transmits externally.

The presence of imported functions such as socket(), connect(), send(), recv(), and calls into WinInet/WinHTTP (or libcurl on Linux builds) is strong static evidence that these modules exist even before dynamic tracing confirms their behaviour.

2. What footprinting techniques are implemented in the executable?

Based on the functions identified above, the binary implements a combination of passive and active footprinting techniques:

- Passive footprinting: WHOIS lookups, DNS record enumeration (A, MX, NS, TXT), and possibly scraping of public sources (search engines, social media, job postings) for organisational and employee data.
- Active footprinting: direct DNS resolution against the target's authoritative servers, ICMP/TCP-based host discovery (ping sweep), and port scanning/banner grabbing to identify live hosts and running services.
- Network topology mapping: traceroute-style hop analysis (use of TTL manipulation) to infer the network path and possibly the presence of firewalls or load balancers.

Together these correspond to the classic footprinting phases taught in the CO2 syllabus: open-source intelligence gathering, DNS interrogation, and network/host discovery — all performed without requiring direct authentication to the target.

3. How can the information collection process be disabled or modified?

- Static patching: locate the call sites to the networking functions (e.g., the CALL instructions to connect()/DnsQuery() identified during disassembly) and NOP them out, or redirect the jump so the corresponding module is skipped.
- Configuration control: if the tool reads a config file or command-line flags to decide which modules run, those modules can be disabled by editing the configuration rather than the binary itself.
- Runtime interception: use API hooking (e.g., Frida, Detours) to intercept and block the relevant WinSock/WinINet calls so no packets actually leave the host, which is useful for safely observing behaviour in a sandbox.
- Network-level mitigation: even without modifying the binary, egress filtering, DNS sinkholing, and blocking outbound WHOIS/port-scan traffic at the firewall neutralise the tool's ability to reach external targets.
- From a defensive standpoint, the safest ‘modification' is containment — running the sample only inside an isolated analysis VM with no route to production networks.

Task 2: DNSScan.exe – DNS Zone Transfer Analysis

DNSScan.exe is suspected of performing DNS zone-transfer (AXFR) requests against a target domain. Reverse engineering focuses on identifying the DNS query construction logic and confirming whether a zone-transfer query type is used.

1. Which function is responsible for initiating DNS requests?

A function typically named SendDNSQuery(), QueryNameServer(), or ResolveDomain() is responsible for constructing and dispatching DNS packets. In the disassembly this function usually:

- Builds a raw DNS query packet header (transaction ID, flags, QDCOUNT) and appends the query name and type.
- Opens a UDP socket to port 53 of a name server for standard queries, and a TCP socket to port 53 for larger responses/zone transfers (TCP is mandatory for AXFR because the response can exceed the UDP size limit).
- Calls low-level socket APIs (socket(), connect(), send(), recv()) — the use of TCP rather than UDP for a DNS-related call is itself a strong indicator worth flagging during analysis.

2. Does the application attempt an AXFR (Zone Transfer) request? Justify your answer.

Yes — the behaviour is consistent with an AXFR attempt, based on the following evidence typically found during static and dynamic analysis:

- String analysis reveals literal DNS type constants or the string "AXFR", and the query-type field in the constructed packet is set to 252 (0xFC), which is the DNS resource-record type code reserved for a full zone transfer.
- The transport layer used is TCP on port 53, not UDP — AXFR is defined to run over TCP because zone data is usually larger than a single UDP datagram.
- Dynamic (sandboxed) execution shows the tool connecting directly to the domain's authoritative name servers (rather than a public recursive resolver), which matches AXFR behaviour since only authoritative/secondary servers are expected to answer zone-transfer requests.
- The response-parsing routine iterates over multiple resource records of mixed types (A, MX, NS, CNAME, SOA) in a loop, which is characteristic of parsing an entire zone file rather than a single-record answer.

3. What information can an attacker obtain using the extracted DNS records?

- A complete internal and external map of the domain's hosts: subdomain names, associated IP addresses, and server roles inferred from hostnames (e.g., vpn, mail, db, dev, backup).
- Mail infrastructure details from MX and TXT records, useful for planning phishing or SPF/DKIM bypass attempts.
- Name-server and SOA information revealing hosting providers, administrative contact addresses, and DNS zone serial/refresh values.
- Exposure of infrastructure that was not meant to be public (staging or internal servers accidentally left in the zone), which significantly expands the organisation's attack surface for follow-up reconnaissance, port scanning, and targeted exploitation.

A successful zone transfer is considered a critical misconfiguration; the standard remediation is to restrict AXFR to explicitly authorised secondary name-server IP addresses (using ACLs such as BIND's allow-transfer directive) and to monitor for AXFR attempts in DNS server logs.

Task 3: IntelGather.exe – OSINT / Competitive Intelligence Tool

IntelGather.exe collects publicly available information — employee names, email addresses, and technology stacks — associated with a target organisation. This corresponds to open-source intelligence (OSINT) gathering, one of the core footprinting activities in the CO2 syllabus.

1. What types of information are collected by the application?

- Employee data: names, job titles, and organisational structure, typically scraped from professional networking pages and company ‘About Us' or ‘Team' pages.
- Email address patterns: harvested directly or inferred by combining employee names with the organisation's known email-naming convention (e.g., first.last@company.com).
- Technology fingerprints: web-server banners, JavaScript library versions, CMS platform signatures, and DNS/WHOIS metadata that reveal the organisation's technology stack and hosting providers.
- Document metadata: author names, software versions, and internal usernames extracted from publicly posted files (PDFs, Office documents) — a classic metadata-leakage technique.
- Social media and public post content that may reveal project names, partnerships, or upcoming events useful for pretexting.

2. Which external websites or services are accessed during execution?

Dynamic analysis (network capture during sandboxed execution) of such a tool commonly shows outbound HTTP/HTTPS requests to:

- Search engines and their APIs, used for automated ‘dorking' queries against the target domain.
- Professional networking and social media sites, scraped for employee and organisational data.
- Public code-hosting platforms, checked for leaked credentials, internal tooling, or configuration files committed by employees.
- Domain intelligence services (WHOIS/DNS lookup APIs, certificate-transparency logs) used to enumerate subdomains and historical infrastructure.
- Job-posting sites, which are frequently mined because job descriptions often reveal the internal technology stack in detail.

3. How can the collected intelligence be misused by attackers?

- Spear-phishing and Business Email Compromise: accurate employee names, titles, and email-format patterns let an attacker craft highly convincing, personalised phishing messages.
- Social engineering / pretexting: knowledge of internal reporting structure and ongoing projects enables an attacker to impersonate a colleague or vendor convincingly over phone or email.
- Targeted exploitation: identified technology stack and software versions let an attacker select known exploits or vulnerabilities specific to that stack rather than attacking blindly.
- Credential-stuffing preparation: harvested email addresses can be checked against known breach databases to find reused passwords.
- Physical and insider-threat planning: organisational charts and employee locations can support impersonation badges, tailgating, or insider-recruitment attempts.

Task 4: MailCraft.exe – Phishing Email Generation Tool

MailCraft.exe automatically generates convincing phishing emails that imitate legitimate organisations. Reverse engineering focuses on the logic that builds and personalises these messages.

1. Which functions are responsible for generating phishing emails?

- TemplateLoader() – reads pre-built HTML/text email templates (often bundled as embedded resources or downloaded at runtime) that mimic the visual branding of a legitimate company.
- PersonalizeMessage() / MergeFields() – performs placeholder substitution, inserting the recipient's name, company, or other harvested details into the template (similar to a mail-merge routine).
- PayloadEmbedder() – inserts a malicious or credential-harvesting link (frequently disguised with URL shorteners or look-alike domains) into the body of the email.

- SendMail() – uses SMTP libraries (raw socket calls to port 25/587, or calls into a mail-sending API) to dispatch the crafted messages, sometimes through compromised or spoofed sender accounts to bypass basic email-authentication checks.

2. What information is used to personalize the messages?

- Recipient name, job title, and department, typically drawn from the OSINT/employee data harvested by a companion reconnaissance module (see Task 3).
- The target organisation's branding: logos, colour schemes, and email signature formats scraped from the real company's website or previous legitimate emails.
- Contextual details such as recent company news, software the organisation uses, or ongoing projects, which make pretext scenarios (e.g., a fake ‘password reset' or ‘invoice' email) more believable.
- The organisation's actual email-domain naming convention, allowing the sender address to closely resemble (spoof or typosquat) a legitimate internal address.

3. How can the phishing functionality be prevented or removed?

- Binary-level: disable or patch the SendMail() routine so crafted messages cannot be transmitted, and strip or quarantine embedded templates/payload resources found in the binary.
- Email-authentication controls: enforce SPF, DKIM, and DMARC on the organisation's mail domain so spoofed sender addresses are rejected or flagged before reaching inboxes.
- Gateway-level filtering: deploy anti-phishing email-security gateways that inspect for look-alike domains, suspicious attachments, and known malicious links.
- User awareness: regular phishing-simulation training reduces the likelihood that a crafted message succeeds even if it is delivered.
- Endpoint controls: application allow-listing and EDR monitoring prevent MailCraft.exe (or similar tools) from executing or reaching outbound SMTP services in the first place.

Task 5: PortalClone.exe – Credential-Harvesting Fake Login Page

PortalClone.exe imitates the login page of a popular online service to capture user credentials. This is a classic phishing/credential-harvesting payload, and reverse engineering focuses on how it captures, stores, and exfiltrates the data.

1. How does the application capture usernames and passwords?

- The tool serves (or embeds) an HTML form that is a pixel-for-pixel visual clone of the legitimate service's login page, hosted either locally via an embedded lightweight web server or on an external look-alike domain.

- A form-submission handler (e.g., an onSubmit JavaScript event or a server-side POST handler such as HandleLogin()) intercepts the entered username and password before — or instead of — forwarding the user to the real service.
- In some variants, keylogging or input-hooking functions (SetWindowsHookEx on Windows, or JavaScript keydown listeners) are used as a fallback to capture keystrokes directly from the fake form fields.

2. Where are the captured credentials stored or transmitted?

- Locally: written to a hidden log file or a local SQLite/text database on disk, often in a temp or AppData-style directory to reduce visibility to the user.
- Remotely: exfiltrated via an HTTP POST request to an attacker-controlled server, or sent as an outbound email/SMTP message; some samples use Telegram-bot APIs or other webhook-style services to blend malicious traffic with legitimate-looking HTTPS calls.
- To evade simple network-based detection, transmitted data is sometimes base64-encoded or lightly encrypted before being sent, and destination domains are frequently freshly registered or use dynamic-DNS providers.

3. What security risks arise from the use of such applications?

- Direct account compromise: harvested credentials give an attacker immediate access to the victim's real account on the legitimate service, including any linked financial or organisational data.
- Credential reuse impact: because many users reuse passwords, a single captured credential set can be leveraged against multiple unrelated services (credential stuffing).
- Foothold for further attack: if the cloned portal targets a corporate SSO or VPN login, captured credentials can provide an initial foothold for lateral movement inside the organisation's network.
- Erosion of trust and brand damage: users who fall victim associate the compromise with the impersonated brand, causing reputational harm to the legitimate service.
- Mitigation: enforce multi-factor authentication (so a captured password alone is insufficient), deploy browser/URL-reputation protections and anti-phishing toolbars, monitor for newly registered look-alike domains, and educate users to verify the URL and TLS certificate before entering credentials.

Code of Conduct

I, AJAY U 192421237 (Name / Reg. No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AJAYYYY U